Handling vulnerable third-party dependencies

Source: https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Vulnerable_Dependency_Management_Cheat_Sheet.md

Objectives in the next 15 minutes

- Show to you why it's important to track third-party dependency regarding their exposure to vulnerabilities.

- Present to you the key points of the management of a vulnerable third-party dependency.

For the technical part, the OWASP cheat sheet, on which is based this presentation, will do the rest and in a regular up-to-date way…

What is a third-party dependency?

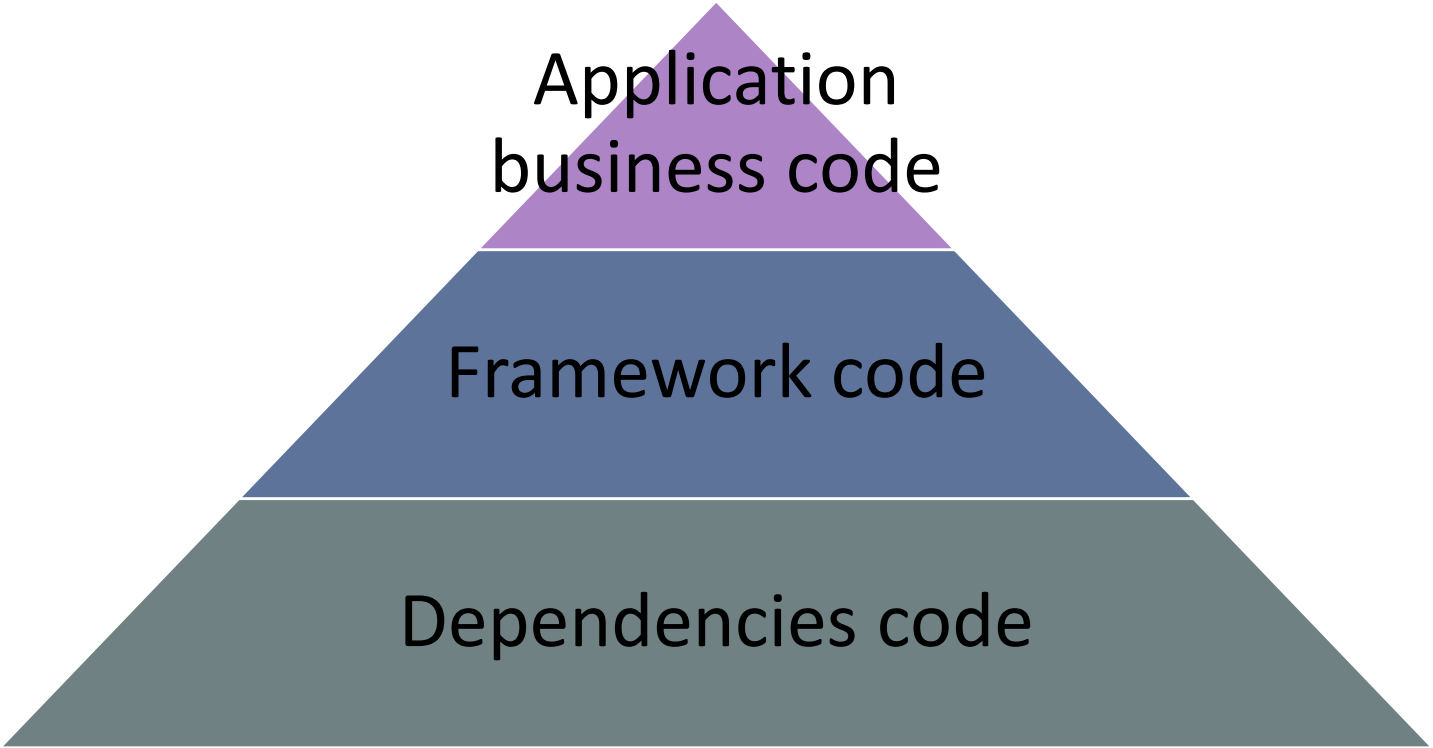
- A third-party dependency is a component (ex: library) used by the application that is external to the company.
- Can be free or commercial.
- Can be closed or open source.

In the rest of this presentation, the term **dependency** will be used refer to **third-party dependency**…

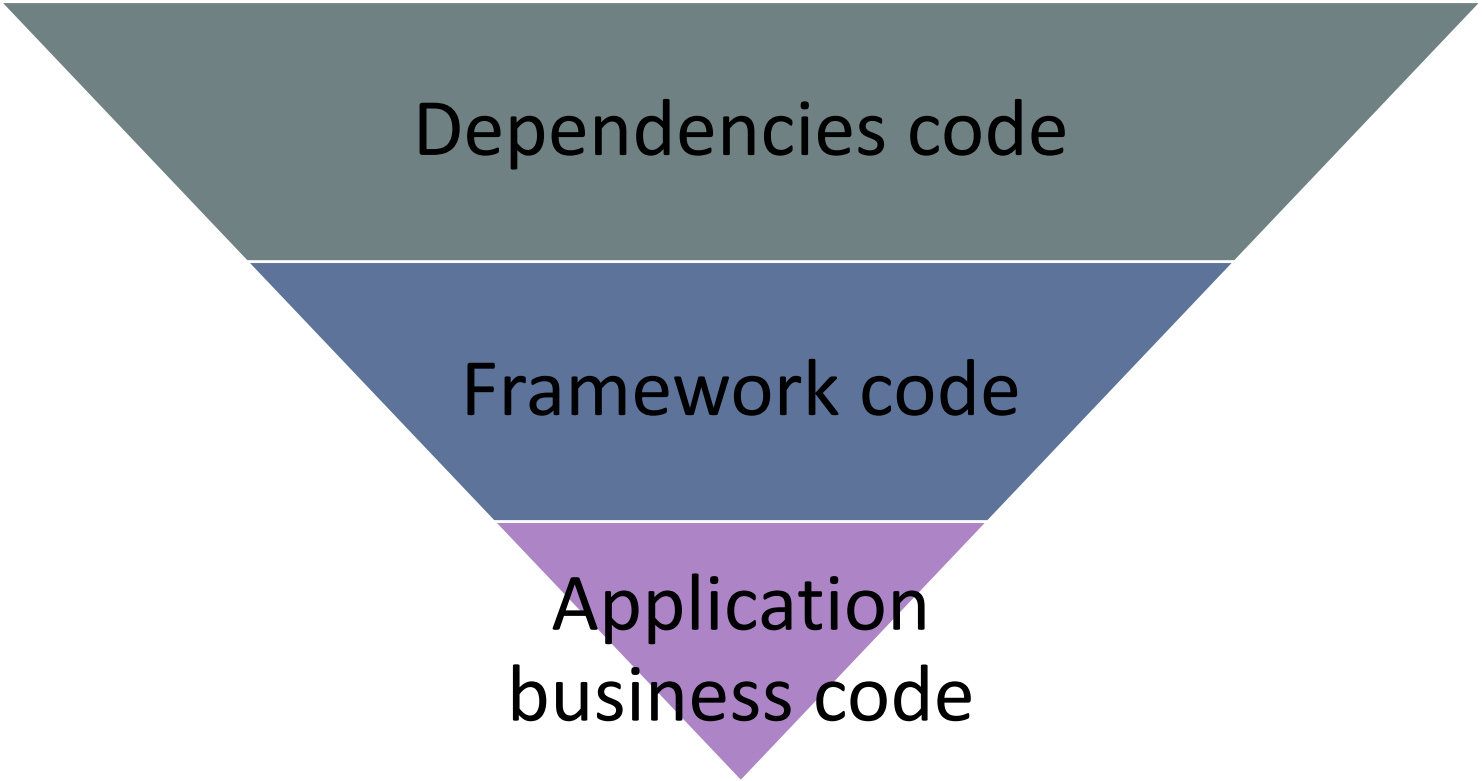
Context

- Most of the projects use dependencies to delegate handling of different kinds of operations, e.g. generation/parsing of documents in a specific format, etc.
- Allows the development team to focus on the real application code supporting the expected business feature.
- The dependency brings forth an expected downside where the security posture of the real application is **now resting on it**.

Application stack

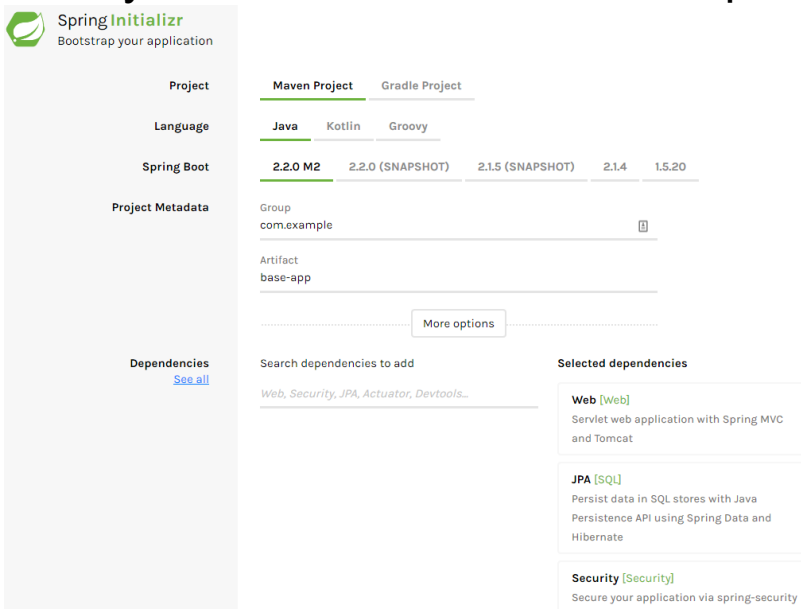


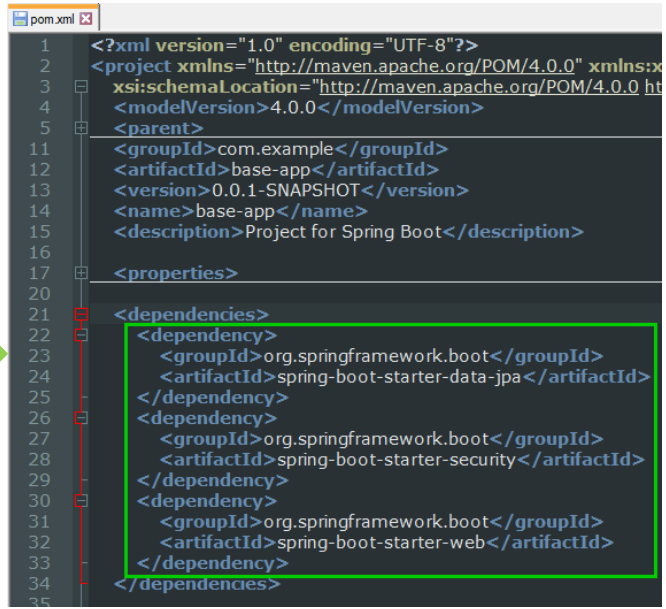
Application stack attack surface



Myth vs reality with dependencies

× ‘I only use a limited number of dependencies…’





Myth vs reality with dependencies

```
--- maven-dependency-plugin:3.1.1:copy-dependencies (default-cli) @ base-app ---
Copying spring-boot-starter-data-jpa-2.2.0.M2.jar to C:\Users\drighetto\Downloads
Copying spring-boot-starter-aop-2.2.0.M2.jar to C:\Users\drighetto\Downloads\demo
Copying aspectjweaver-1.9.3.jar to C:\Users\drighetto\Downloads\demo\target\depen
Copying spring-boot-starter-jdbc-2.2.0.M2.jar to C:\Users\drighetto\Downloads\dem
```

```
drighetto@ MINGW64 ~/Downloads/demo/target/dependency
$ ls *.jar | wc -l
68
```

68 dependencies used by this simple base project without any business code…

Myth vs reality with dependencies

Unexpected gift:

A scan of the dependencies with the tool OWASP Dependency Check shown that there is one dependency vulnerable in this freshly generated base app:

🎁 CVE-2019-0232 impacting the *CGI Servlet in Apache Tomcat*.

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence
tomcat-embed-core-9.0.17.jar	cpe:2.3:a:apache:tomcat:9.0.17:*:*:*:*:*:*:* cpe:2.3:a:apache_software_foundation:tomcat:9.0.17:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.17:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.17	HIGH	1	Highest

CVSS v2.0 Severity and Metrics:

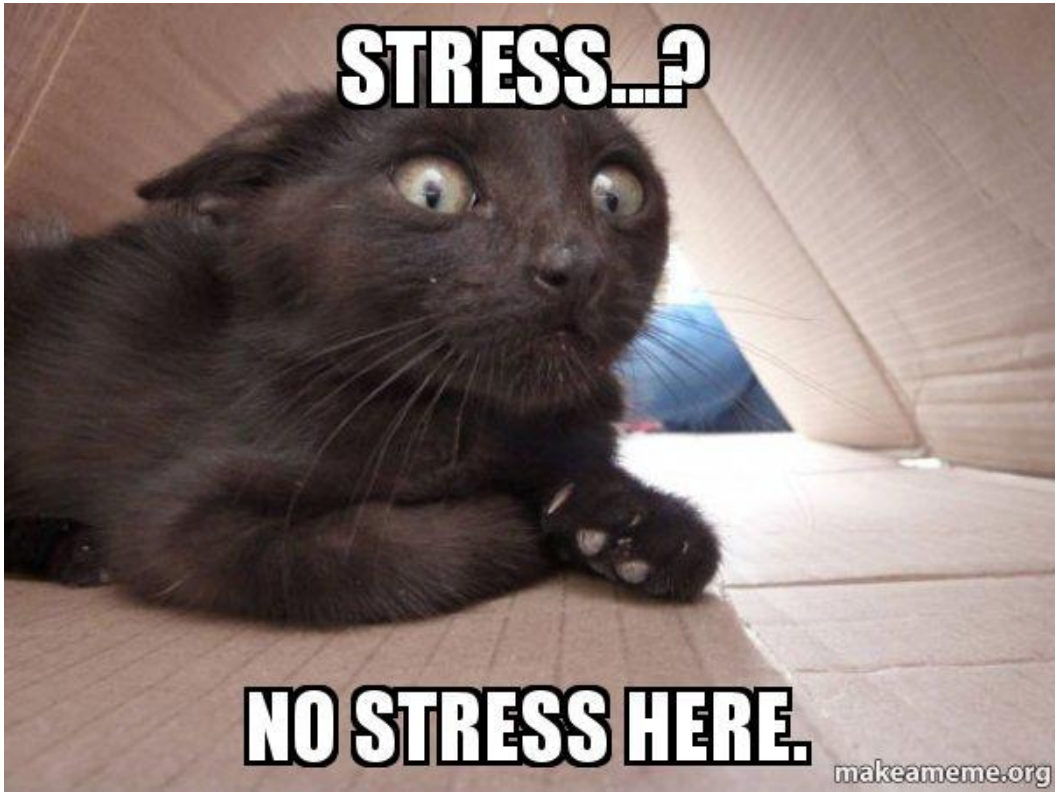
Base Score: 9.3 HIGH

Vector: (AV:N/AC:M/Au:N/C:C/I:C/A:C) (V2 legend)

Impact Subscore: 10.0

Exploitability Subscore: 8.6

Myth vs reality with dependencies



Don’t panic or stress your dev/sec team!

- × It’s not because you have a vulnerable dependency that your application is vulnerable!
- × Give time to the development/security team to analyse the issue and identify if vulnerable code/function/feature is used by the application.
- × For example, for the issue on Tomcat seen in previous slide: The CVE mention that ‘*The CGI Servlet is disabled by default*’ so **unless you have enabled it explicitly**, you are not exposed to the CVE, so, you have time to patch…

Remark about the detection by tools

- Tools can only detect a vulnerable dependency in the following conditions:
  - A CVE is published about the dependency.
  - It uses an internal vulnerability databases filled from different sources.
  - It monitors sites like Exploit-DB, Full Disclosure … to spot published vulnerability (and exploit) on dependency that has not necessarily a CVE created for it.

- If you decide to select a tool then **ensure to test the reliability/quality of the vulnerability database of the evaluated tools**, especially, regarding a vulnerability without CVE…

Remark about the detection by tools

× **Never trust** commercial speech or demo ➔ Test the tool yourself in your context with your evaluation tests plan!

This is an example of an evil test (**do not inform the vendor about this test**):

1. Import a dependency in your test project for which an exploit has been published on Exploit-DB or Full Disclosure and there is not CVE created.

2. Run a scan of the project's dependencies each day during one week to see when the vulnerability will be added into the database of vulnerabilities of the evaluated tools (**vulnerability can perhaps never be added**).

× Using this test, you will identify the reaction delay and detection capacity.

Importance of the test suites…

- Whatever the situation of patching you will face, you should have a foundation of automated test suites (unit, functional, integration) for the application in order to:
  - Ensure that the application is still functional from a technical and business point of view after the patching.
  - Ensure that other modules using the application (if this one is a library) are still functional from a technical and business point of view after the patching.
  - Quickly validate different versions of the patched dependency.

Patching cases

Context of the patching case	Effort	Documentation
Patched version of the component has been released by the provider.	⭐	Case 1
Provider informs the team that it will take a while to fix the issue and, so, a patched version will not be available before months.	⭐⭐	Case 2
Provider inform the team that he cannot fix the issue, so no patched version will be released at all (applies also if provider does not want to fix the issue or does not answer at all). • In this case the only information given to the development team is the CVE.	⭐⭐⭐	Case 3
The vulnerable dependency is found during one of the following situation in which the provider is not aware of the vulnerability: • Via the discovery of a full disclosure post on the Internet. • During a penetration test.	⭐⭐	Case 4

× For each case, the documentation link provides the process to follow to apply the patching.
× Effort cost is rated from one to three stars where one star is easy and three stars is really hard.

Tools

Name	Free	Technologies
OWASP Dependency Check	Yes	Full support for Java and .NET Experimental support: Python, Ruby, PHP (composer), NodeJS, C, C++.
NPM Audit	Yes	Full support: NodeJS, Javascript.
Snyk	Partial	Full support for many languages and package manager. Can indicate if the vulnerable dependency is used in the code.
JFrog XRAY	No	Full support for many languages and package manager.
Renovate	No	Full support for many languages and package manager.
Requires.io	Partial	Allow to detect old dependencies - Python only.

Tools in the table above are ones that have been used on customers or on during the spare time for personal projects.

Thank you

Source: https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Vulnerable_Dependency_Management_Cheat_Sheet.md